Machine Learning

Multinomial Logistic Regression: Classifying More Than Two Classes

Dr. G. Omprakash

KLEF Hyderabad (Aziz Nagar) – Dept. of ECE

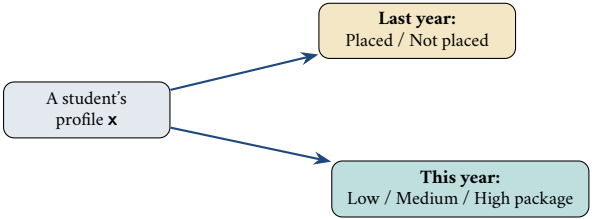
Machine Learning (25SC2107E)

Content

- Why two classes are easy and three classes are not.
- **Softmax**: logistic regression with more than two classes.
- One-vs-Rest versus multinomial, and how `scikit-learn` chooses.
- Categorical cross-entropy, and what the decision regions look like.
- Precision, recall and F1, and why accuracy alone misleads.

Multinomial Logistic Regression

Scenario

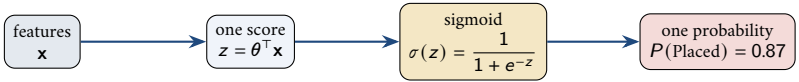


two classes $\Rightarrow$ sigmoid

three classes $\Rightarrow$?

- The features have not changed. The shape of the answer has changed.
- A yes/no model returns one number. A three-way model must return three.

Binary logistic regression, which you already know

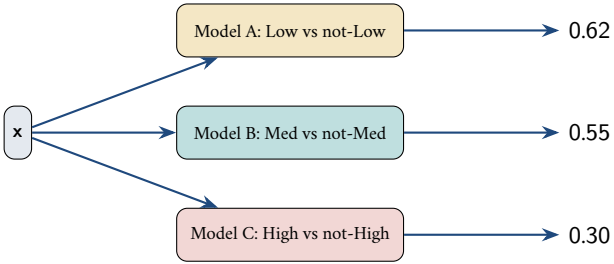


- One set of weights. One score. One probability.
- `coef_` therefore has exactly **one row**.
- That works only because with two classes, knowing one probability tells you the other.

Pause & Think

With three classes — `Low`, `Medium`, `High` — one number can no longer describe the answer. What is the smallest change that would fix this?

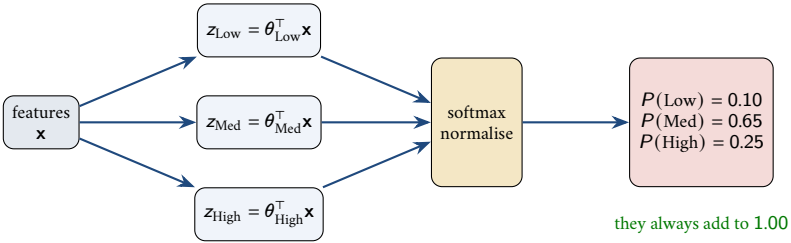
First attempt: One-vs-Rest (three separate binary models)



total = 1.47
not a probability

- Each model is trained **separately** and never knows the others exist.
- Their answers therefore do not have to add up to 1 — and generally do not.
- You can rescale them afterwards, but the rescaling was never part of the training.

Softmax — one score per class, then normalise



$$P(\text{class } k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$ and the prediction is the largest of them

Key Insight

Softmax is simply the sigmoid generalised to K classes. With $K = 2$ the two are identical, which is why one command handles both.

Worked example

Suppose one student's three scores come out as $z_{\text{Low}} = 0.5$, $z_{\text{Med}} = 2.0$, $z_{\text{High}} = 1.0$.

Class	z_k	e^{z_k}	$\hat{p}_k$
Low	0.5	1.65	0.14
Medium	2.0	7.39	**0.63**
High	1.0	2.72	0.23
		11.76	1.00

1. Exponentiate each score: kills negatives, keeps the order.
2. Add them: $1.65 + 7.39 + 2.72 = 11.76$.
3. Divide each by the total: e.g. $7.39/11.76 = 0.63$.
4. Predict the largest: **Medium**.

Pause & Think

The gap between Medium and High was 1.0 in score but became 0.63 vs 0.23 in probability — almost 3×. Why does exponentiating amplify gaps?

Why the exponential?

- **Positivity.** $e^z > 0$ always, so no negative "probabilities". Dividing raw scores by their sum would fail the moment one score is negative.
- **Order preserved.** e^z is increasing, so the biggest score stays the biggest probability.
- **Differentiable.** Gradient descent needs a smooth function; a hard "pick the max" rule has no useful gradient.
- **Only differences matter.** Adding the same constant c to every score changes nothing.

$$\frac{e^{z_k+c}}{\sum_j e^{z_j+c}} = \frac{e^c e^{z_k}}{e^c \sum_j e^{z_j}} = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Name check

"Softmax" = a soft version of max: instead of giving the winner all the mass, it gives the winner most of it and leaves the rest a share.

Check: with $K = 2$, softmax is the sigmoid

Take two classes with scores z_1 and z_0:

$$\hat{p}_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_0}} = \frac{1}{1 + e^{-(z_1 - z_0)}} = \sigma(z_1 - z_0)$$

- Divide top and bottom by e^{z_1} — that is the whole derivation.
- The binary model never needed two weight vectors, only their **difference**. That is exactly the redundancy from the previous slide.

Key Insight

So logistic regression was never a separate method. It is the $K = 2$ case of one general model, which is why `LogisticRegression()` handles both without you changing a single argument.

The label side: one-hot encoding

y	$\mathbf{y}$ (one-hot) [Low, Med, High]	model output $\hat{\mathbf{p}}$
Medium	(0, 1, 0)	(0.19, 0.63, 0.18)
High →	(0, 0, 1)	(0.10, 0.30, 0.60)
Low	(1, 0, 0)	(0.55, 0.35, 0.10)

- The true label becomes a vector with a single 1 in the correct slot.
- Now the truth and the prediction have the **same shape**, so we can compare them directly.
- `scikit-learn` does this internally — you still pass plain text labels in $\mathbf{y}$.

The loss: categorical cross-entropy

$$J(\Theta) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik} \ln \hat{p}_{ik} \overset{\text{one-hot}}{=} -\frac{1}{n}\sum_{i=1}^{n} \ln \hat{p}_{i,\,\text{true class}}$$

- Because y_i is one-hot, every term except the true class multiplies by zero.
- So the loss reads: how much probability did you give the right answer?
- Confident and right $\Rightarrow \ln(0.95) = -0.05$, tiny loss.
- Confident and wrong $\Rightarrow \ln(0.02) = -3.9$, huge loss.

$\hat{p}_{\text{true}}$	$-\ln \hat{p}$	verdict
0.99	0.01	excellent
0.63	0.46	acceptable
0.33	1.11	no better than guessing
0.02	3.91	badly wrong

Key Insight

Cross-entropy punishes **confident mistakes** far more than hesitant ones. That is what pushes the model to be honest about uncertainty.

The gradient has not changed shape

$$\frac{\partial J}{\partial \Theta} = \frac{1}{n} \mathbf{X}^\top \left(\hat{\mathbf{P}} - \mathbf{Y} \right) \qquad \Theta \leftarrow \Theta - \eta \frac{\partial J}{\partial \Theta}$$

- $\hat{\mathbf{P}} - \mathbf{Y}$ is simply **prediction minus truth**, exactly as in linear and binary logistic regression.
- Only the dimensions grew: $\mathbf{Y}$ and $\hat{\mathbf{P}}$ are $n \times K$, so Θ is $d \times K$.
- The same gradient descent loop from the previous session runs unchanged.

Practical reminders

Scale your features — softmax fitting uses gradient descent, so unscaled columns make it crawl.

If you see a `ConvergenceWarning`, raise `max_iter` (or scale better).

Key Insight

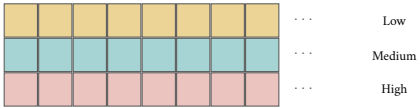
Three models so far, one update rule. The only thing that ever changes is how $\hat{\mathbf{P}}$ is produced from $\mathbf{X}$.

How to tell which one you got: look at `coef_`

Binary: `coef_.shape = (1, 15)`

one row = one set of weights

Multinomial: `coef_.shape = (3, 14)`



three rows = one set of weights per class

- You write `LogisticRegression()` in both cases. Scikit-learn counts the distinct values in y and chooses for you.
- The number of **rows** in `coef_` is therefore the evidence of what it chose.
- Each row answers a separate question: how much does each feature push a student towards this class?

In code: nothing new to type

Binary — y has 2 labels

```
from sklearn.linear_model \
     import LogisticRegression

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)

clf.coef_.shape
# (1, 15)
clf.classes_
# ['NotPlaced', 'Placed']
clf.predict_proba(X_test[:1])
# [[0.13, 0.87]]
clf.predict(X_test[:1])
# ['Placed']
```

Multinomial — y has 3 labels

```
from sklearn.linear_model \
     import LogisticRegression

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)

clf.coef_.shape
# (3, 14)
clf.classes_
# ['High', 'Low', 'Medium']
clf.predict_proba(X_test[:1])
# [[0.23, 0.14, 0.63]]
clf.predict(X_test[:1])
# ['Medium']
```

Key Insight

Same code both sides. Only the width of the output changed — decided by y.

Metrics

The confusion matrix

	predicted NotPlaced	predicted Placed
truly NotPlaced	**TN** correctly said NotPlaced	**FP** wrongly said Placed
truly Placed	**FN** wrongly said NotPlaced	**TP** correctly said Placed

$$\text{Precision} = \frac{TP}{TP + FP}$$ when it says Placed, is it right?

$$\text{Recall} = \frac{TP}{TP + FN}$$ of all who were Placed, how many found?

$$F_1 = 2 \cdot \frac{\text{Prec} \times \text{Rec}}{\text{Prec} + \text{Rec}}$$

Key Insight

Accuracy is $\frac{TP + TN}{\text{everything}}$. It collapses all four boxes into one number, and so cannot tell you which kind of mistake the model is making.

With three classes the matrix is 3×3

true \ pred	Low	Med	High
Low	**140**	25	5
Med	30	**210**	40
High	4	35	**111**

- The diagonal is correct; everything off it is a specific, nameable error.
- Precision for a class = its diagonal ÷ its **column** total.
- Recall for a class = its diagonal ÷ its **row** total.

Read the off-diagonal too

Med→High (40) and High→Med (35) dominate the errors.

The model confuses the two adjacent bands and almost never confuses Low with High — which tells you the errors are mild, something accuracy alone could never say.

The imbalance trap, with the tutorial's own numbers

Class	Prec.	Recall	F1
Placed (70%)	0.83	0.90	**0.87**
NotPlaced (30%)	0.72	0.58	**0.64**
Overall accuracy	**0.81**		

- The single figure 0.81 looks respectable.
- It conceals that the model finds only **58%** of the students who were not placed.

Why this is the wrong way round

The **smaller** class is nearly always the one you care about:

students at risk of not being placed, fraudulent transactions, diseased patients.

A model that is excellent on the majority and poor on the minority is **precisely backwards** for the purpose it was built for.

Key Insight

Always print the **per-class** report. Accuracy is a summary, and summaries hide exactly the thing you need to see.

MCQs (1/2)

- You fit `LogisticRegression` and `coef_.shape` is `(4, 20)`. The task had
  (a) 4 features (b) 4 classes (c) 20 classes (d) 2 classes

MCQs (1/2)

1. You fit `LogisticRegression` and `coef_.shape` is `(4, 20)`. The task had
   (a) 4 features (b) 4 classes (c) 20 classes (d) 2 classes

   Answer: (b)

2. Softmax probabilities across all classes always
   (a) equal each other (b) sum to 1 (c) sum to the number of classes (d) are all above 0.5

MCQs (1/2)

1. You fit `LogisticRegression` and `coef_.shape` is `(4, 20)`. The task had
   (a) 4 features (b) 4 classes (c) 20 classes (d) 2 classes

   Answer: (b)

2. Softmax probabilities across all classes always
   (a) equal each other (b) sum to 1 (c) sum to the number of classes (d) are all above 0.5

   Answer: (b)

3. Adding the same constant to every score z_k changes the softmax output
   (a) not at all (b) a little (c) completely (d) only if the constant is negative

MCQs (1/2)

1. You fit `LogisticRegression` and `coef_.shape` is `(4, 20)`. The task had
   (a) 4 features (b) 4 classes (c) 20 classes (d) 2 classes

   Answer: (b)

2. Softmax probabilities across all classes always
   (a) equal each other (b) sum to 1 (c) sum to the number of classes (d) are all above 0.5

   Answer: (b)

3. Adding the same constant to every score z_k changes the softmax output
   (a) not at all (b) a little (c) completely (d) only if the constant is negative

   Answer: (a)

4. With $K = 3$, the model predicts the class with
   (a) probability above 0.5 (b) the largest probability (c) the largest coefficient (d) the smallest loss

MCQs (1/2)

1. You fit `LogisticRegression` and `coef_.shape` is `(4, 20)`. The task had
   (a) 4 features (b) 4 classes (c) 20 classes (d) 2 classes

   Answer: (b)

2. Softmax probabilities across all classes always
   (a) equal each other (b) sum to 1 (c) sum to the number of classes (d) are all above 0.5

   Answer: (b)

3. Adding the same constant to every score z_k changes the softmax output
   (a) not at all (b) a little (c) completely (d) only if the constant is negative

   Answer: (a)

4. With $K = 3$, the model predicts the class with
   (a) probability above 0.5 (b) the largest probability (c) the largest coefficient (d) the smallest loss

   Answer: (b)

MCQs (2/2)

- The boundary between two classes in multinomial logistic regression is
  (a) a curve (b) a straight line/hyperplane (c) a circle (d) undefined

MCQs (2/2)

- The boundary between two classes in multinomial logistic regression is
  (a) a curve (b) a straight line/hyperplane (c) a circle (d) undefined

Answer: (b)

- Categorical cross-entropy for one sample reduces to
  (a) $-\ln \hat{p}_{\text{true}}$ (b) $\sum_k \hat{p}_k$ (c) $(y - \hat{p})^2$ (d) $\ln \sum_k e^{z_k}$

MCQs (2/2)

- The boundary between two classes in multinomial logistic regression is
  (a) a curve (b) a straight line/hyperplane (c) a circle (d) undefined

Answer: (b)

- Categorical cross-entropy for one sample reduces to
  (a) $-\ln \hat{p}_{\text{true}}$ (b) $\sum_k \hat{p}_k$ (c) $(y - \hat{p})^2$ (d) $\ln \sum_k e^{z_k}$

Answer: (a)